

Laboratory Practice – III

Course code: MCA10602

Assignment 1 – STQA - CO607.1

1. Objective

To understand and implement the process of **Software Testing** by creating a structured **Test Plan**, designing **Test Cases**, and documenting defects in a **Defect Report** for an Online Banking Application.

2. Introduction

Software Testing and Quality Assurance (STQA) ensures that software applications are reliable, secure, and function according to requirements. An **Online Banking Application** is a critical real-time system where issues like incorrect transactions, failed authentication, or security breaches can cause severe financial and reputational damage.

To ensure software quality, testers follow a systematic approach:

- **Test Plan** defines the scope, objectives, resources, and schedule of testing activities.
- **Test Cases** describe step-by-step scenarios with input, action, and expected output to validate functionality.
- **Defect Report** records deviations from expected behavior, helping developers track and fix issues efficiently.

This assignment demonstrates the application of STQA concepts by preparing a test plan, test cases, and defect report for an Online Banking Application.

3. Tools/Materials Used: SRS, Testing documentation template, Microsoft Word/Excel

4. Procedure (Pseudocode/Algorithmic Format)

Step 1: Analyze the requirements of the Online Banking Application.

Step 2: Identify core modules (e.g., Login, Fund Transfer, Balance Enquiry, Transaction History).

Step 3: Prepare the Test Plan including:

- Scope and objectives of testing
- Testing strategy and levels (unit, integration, system, acceptance)
- Features to be tested and not to be tested
- Test environment setup
- Resources and schedule

Step 4: Design Test Cases in tabular form with fields: Test ID, Test Description, Preconditions, Test Steps, Expected Result, Actual Result, Status.

Step 5: Execute the Test Cases on the application.

Step 6: Document any deviations from expected results in a Defect Report with: Defect ID, Module, Severity, Steps to Reproduce, Expected vs. Actual Result, Status.

Step 7: Review and finalize the Test Plan, Test Cases, and Defect Report for submission.

5. Results

- A detailed **Test Plan** was prepared covering scope, strategy, features, and risks of testing for the Online Banking Application.
- A set of **Test Cases** was created to validate functionalities such as Login authentication, Fund Transfer correctness, Balance Enquiry accuracy, and Transaction History retrieval.
- A **Defect Report** was generated highlighting defects such as incorrect error messages, failed login attempts, or transaction mismatches.
- The exercise demonstrated how systematic testing improves application reliability, accuracy, and user confidence in critical applications like online banking.

Assignment 2 – STQA - CO607.1

1. Objective

To design and analyze **black box test cases** for verifying bank transaction charges using **Equivalence Class Partitioning (ECP)** and **Boundary Value Analysis (BVA)** techniques.

2. Introduction

Black box testing is a software testing technique where the functionality of the system is tested without looking into its internal code or logic. Two important strategies in black box testing are:

- **Equivalence Class Partitioning (ECP):** Divides input data into partitions (classes) where test cases are designed to cover one representative value from each class. This reduces redundancy while ensuring test coverage.
- **Boundary Value Analysis (BVA):** Focuses on values at the edges of input domains since boundary conditions are where errors often occur.

In this assignment, the requirement states that a **bank charges different percentages depending on the transaction amount:**

- **5%** of the amount if transaction ≤ 1000
- **6%** of the amount if $1000 < \text{transaction} \leq 2000$
- **7%** of the amount if transaction > 2000

By applying **ECP and BVA**, we can systematically design test cases to validate correctness of charges across all input ranges.

3. Materials/Tools Used: Requirement specification, Microsoft Excel

4. Procedure (Pseudocode/Algorithmic Format)

Step 1: Read the requirement specification for transaction charges.

Step 2: Define valid input classes based on the conditions:

- Class 1: Transaction ≤ 1000 (5%)
- Class 2: $1001 \leq \text{Transaction} \leq 2000$ (6%)
- Class 3: Transaction > 2000 (7%)

Step 3: Define invalid class for negative transaction amounts.

Step 4: Apply **Equivalence Class Partitioning (ECP)**: Choose one value from each class

Step 5: Apply **Boundary Value Analysis (BVA)**:

- Select values at and around boundaries (e.g., 999, 1000, 1001, 2000, 2001).

Step 6: Document all test cases in Excel sheet

Step 7: Execute the test cases and compare actual charges with expected values.

Step 8: Record any discrepancies in a defect log (if found).

5. Results

- Black box test cases were successfully designed using **ECP and BVA**.
- ECP helped reduce redundant test cases by selecting representative values for each valid and invalid class.
- BVA ensured correctness of charge calculation at critical transaction limits (1000, 1001, 2000, 2001).
- The Excel sheet (attached separately) contains the detailed set of test cases.
- The testing approach validated that the bank's transaction charge logic works correctly across different ranges.

Assignment 5 – DL - CO607A.3

1. Objective

To understand the commonly used **Python libraries for Deep Learning** and implement a simple neural network using one of these libraries.

2. Introduction

Deep Learning (DL) is a subfield of Machine Learning that focuses on multi-layered neural networks capable of learning hierarchical patterns from data. Python has become the most popular programming language for Deep Learning due to its **simplicity, flexibility, and extensive ecosystem of libraries**.

Key Python libraries such as **TensorFlow, Keras, and PyTorch** provide high-level APIs, GPU acceleration, and pre-built functions to simplify neural network design and training. These libraries handle tasks like automatic differentiation, tensor manipulation, optimization, and visualization, allowing researchers and developers to focus on solving problems rather than writing complex mathematical routines from scratch.

In this assignment, we first explore the role of Python libraries for Deep Neural Networks and then build a simple neural network model using them.

3. Materials/Tools Used:

- Python 3.x
- Libraries: TensorFlow/Keras, NumPy, Matplotlib (for visualization)
- Dataset: Simple synthetic dataset (e.g., XOR problem or MNIST digits subset)

4. Procedure (Pseudocode/Algorithmic Format)

Step 1: Import the required Python libraries (TensorFlow/Keras, NumPy, Matplotlib).

Step 2: Load or prepare a simple dataset (e.g., small numeric dataset or MNIST digits).

Step 3: Define a neural network with:

- Input layer (matching dataset features)
- Hidden layer(s) with activation function (e.g., ReLU)
- Output layer with activation (e.g., softmax/sigmoid)

Step 4: Compile the model by specifying optimizer (e.g., Adam), loss function, and metric (e.g., accuracy).

Step 5: Train the model using training data for fixed epochs.

Step 6: Evaluate the model using test data and record accuracy.

Step 7: Visualize training history (loss vs. accuracy graph).

Step 8: Interpret the results and finalize observations.

5. Results

- A simple neural network was successfully implemented in Python using **TensorFlow/Keras**.
- The model was trained and evaluated, producing an accuracy value that demonstrates the learning capability of deep neural networks.
- Visualization of training accuracy and loss confirmed that the model converged during training.
- This experiment highlighted how Python libraries simplify the development of deep neural networks by providing pre-built functions for model design, training, and evaluation.

Assignment 6 – DL - CO607A.3

1. Objective

To design and implement a **Convolutional Neural Network (CNN)** model for classifying images from a given dataset.

2. Introduction

Convolutional Neural Networks (CNNs) are a class of deep neural networks specifically designed for **image processing and classification tasks**. Unlike traditional neural networks, CNNs use **convolutional layers** to automatically extract spatial features such as edges, textures, and shapes from input images. Pooling layers reduce dimensionality while retaining important information, and fully connected layers perform final classification.

CNNs have revolutionized fields like **computer vision, object detection, and medical imaging**, providing state-of-the-art performance in tasks such as handwriting recognition, facial recognition, and autonomous driving. By applying CNNs to an image dataset, we can train the model to recognize distinct patterns and classify images accurately.

3. Materials/Tools:

- Python 3.x, TensorFlow/Keras
- Dataset: Standard image dataset (e.g., MNIST, CIFAR-10, or custom dataset)

4. Procedure (Pseudocode/Algorithmic Format)

Step 1: Import required libraries (TensorFlow/Keras, NumPy, Matplotlib).

Step 2: Load and preprocess the image dataset (resize, normalize pixel values, one-hot encode labels).

Step 3: Define the CNN architecture with layers:

- Convolutional layer(s) with filters and activation (e.g., ReLU)
- Pooling layer(s) for dimensionality reduction
- Flatten layer to convert 2D feature maps into 1D vector
- Fully connected (Dense) layers
- Output layer with softmax activation for classification

Step 4: Compile the model using optimizer (e.g., Adam), loss function (e.g., categorical crossentropy), and metrics (e.g., accuracy).

Step 5: Train the CNN model using the training dataset for fixed epochs and batch size.

Step 6: Evaluate model performance on the test dataset.

Step 7: Visualize training accuracy and loss graphs.

Step 8: Record classification accuracy and analyze the results.

5. Results

- A CNN model was successfully designed and implemented for image classification.
- The model achieved good classification accuracy on the dataset, demonstrating its ability to automatically extract image features.
- Training history graphs confirmed that the CNN converged during learning.
- This assignment showed how CNNs are highly effective in solving real-world image recognition problems.

Assignment 3 – STQA - CO607.2

1. Objective

To install and use **Selenium IDE** for automated testing by recording and replaying a test scenario in the Google Chrome browser.

2. Introduction

Selenium IDE is a lightweight and easy-to-use automation tool for web applications. It works as a browser extension and allows testers to create, record, and replay test scripts without advanced programming knowledge. The tool supports features such as command execution, assertion checks, and exporting test cases to various programming languages.

Automating repetitive test scenarios like **searching content on Google** helps ensure accuracy and saves time compared to manual testing. This assignment demonstrates the use of Selenium IDE to record and execute a test case that searches for “java tutorials” on Google and clicks on a search result link.

3. Materials/Tools:

- Google Chrome browser
- Selenium IDE extension installed

4. Procedure (Pseudocode/Algorithmic Format)

Step 1: Install Selenium IDE extension in Google Chrome.

Step 2: Launch Selenium IDE from the browser extensions menu.

Step 3: Create a new test project and name it (e.g., "Google Search Test").

Step 4: Click on **Record** to begin recording the test scenario.

Step 5: Open the Chrome browser window through Selenium IDE.

Step 6: Navigate to the URL: www.google.com.

Step 7: Click on the **Google Search** text box.

Step 8: Type the value “**java tutorials**” into the search box.

Step 9: Press **Enter** or click the **Search button**.

Step 10: From the displayed results, click on any one of the “Java tutorial” links.

Step 11: Stop recording in Selenium IDE.

Step 12: Replay the recorded test script to verify that the scenario executes successfully.

5. Results

- Selenium IDE was successfully installed and configured in the Chrome browser.
- A test case was recorded that automated the process of searching for “**java tutorials**” on Google.
- The replay feature confirmed that the browser automatically performed all actions (opening Google, entering text, and clicking a tutorial link).
- This assignment demonstrated how Selenium IDE can be used for quick test automation through record and replay functionality.

Assignment 4 – STQA - CO607.2

1. Objective

To automate the **login flow of a web-based email service (Gmail/Rediffmail/Hotmail)** using Selenium WebDriver.

2. Introduction

Selenium WebDriver is one of the most widely used automation tools for testing web applications. Unlike Selenium IDE, which relies on record-and-replay, **WebDriver interacts directly with the browser** through programming languages like Java, Python, or C#. This provides more flexibility, control, and robustness for automating complex scenarios.

The login functionality of webmail applications such as Gmail, Rediffmail, or Hotmail is a critical use case, as it involves **navigating pages, entering credentials, handling buttons, and validating redirections**. Automating this process helps verify that authentication works correctly and consistently across browsers.

This assignment demonstrates how to automate the login process of an email service using Selenium WebDriver.

3. Materials/Tools:

- Google Chrome browser
- ChromeDriver executable (or appropriate browser driver)
- Selenium WebDriver library
- Programming language (e.g., Java/Python)

4. Procedure (Pseudocode/Algorithmic Format)

Step 1: Install Selenium WebDriver and configure it with the programming environment.

Step 2: Download and set up the ChromeDriver (or other browser driver) in the system path.

Step 3: Launch the Chrome browser instance using WebDriver.

Step 4: Open the login page of the chosen email service (e.g., Gmail: <https://mail.google.com>).

Step 5: Locate the **username/email input field** and enter a valid email ID.

Step 6: Click on the **Next/Continue** button.

Step 7: Locate the **password field** and enter the correct password.

Step 8: Click on the **Login/Sign in** button.

Step 9: Wait for the mailbox homepage/dashboard to load.

Step 10: Validate successful login by checking the presence of an element (e.g., inbox, profile icon).

Step 11: Close the browser instance.

5. Results

- Selenium WebDriver successfully launched the browser and navigated to the email login page.
- The script automatically entered login credentials, clicked necessary buttons, and logged into the email account.
- Successful login was validated by confirming the presence of mailbox elements.
- This assignment demonstrated how **Selenium WebDriver enables powerful, code-driven automation** of web application flows such as email authentication.

Assignment 7 – DL - CO607A.4

1. Objective

To design and implement a **Deep Learning-based image classification model** that can recognize and categorize images into predefined classes.

2. Introduction

Image classification is a fundamental problem in **computer vision**, where the goal is to assign a label to an image from a set of predefined categories. It forms the basis of advanced applications like facial recognition, medical image diagnosis, and object detection.

Deep Learning techniques, particularly **Convolutional Neural Networks (CNNs)**, have achieved remarkable performance in this domain by automatically learning hierarchical features such as edges, textures, and shapes directly from raw image pixels. Python libraries such as **TensorFlow** and **Keras** provide high-level APIs that make it easier to build, train, and evaluate classification models efficiently.

This assignment focuses on implementing a simple image classification model that processes input images, extracts features, and classifies them into different categories.

3. Materials/Tools:

- Python 3.x, TensorFlow/Keras
- Dataset: Standard image dataset (e.g., CIFAR-10, MNIST, or custom dataset)

4. Procedure (Pseudocode/Algorithmic Format)

Step 1: Import required libraries (TensorFlow/Keras, NumPy, Matplotlib).

Step 2: Load and preprocess the dataset (resize, normalize, and one-hot encode labels).

Step 3: Define the image classification model architecture:

- Input layer (matching image dimensions)
- Convolutional and pooling layers to extract spatial features
- Flatten layer to convert 2D feature maps to 1D
- Dense layers for learning complex relationships
- Output layer with softmax activation for classification

Step 4: Compile the model with optimizer (e.g., Adam), loss function (categorical crossentropy), and evaluation metric (accuracy).

Step 5: Train the model using training data for fixed epochs and batch size.

Step 6: Evaluate model accuracy and loss using test data.

Step 7: Visualize training history (loss vs. accuracy graphs).

Step 8: Record classification performance and analyze misclassifications if any.

5. Results

- An image classification model was successfully built and trained using a standard dataset.
- The model achieved good accuracy on test images, confirming its ability to generalize patterns from training data.
- Visualization of accuracy and loss trends showed proper convergence during training.
- The assignment demonstrated how Deep Learning can be applied effectively for solving image classification problems in real-world scenarios.

Assignment 8 – DL- CO607A.4

1. Objective

To implement an **autoencoder-based anomaly detection model** that learns normal data patterns and identifies anomalies as deviations from the learned representation.

2. Introduction

Anomaly detection is the task of identifying unusual patterns or outliers in data that do not conform to expected behavior. It has important applications in areas such as **fraud detection, network security, medical diagnosis, and industrial fault detection**.

An **Autoencoder** is a type of neural network designed to learn efficient data representations in an unsupervised manner. It consists of two main components:

- **Encoder:** Compresses input data into a lower-dimensional latent space.
- **Decoder:** Reconstructs the original data from the compressed representation.

When trained on **normal data only**, the autoencoder learns to reconstruct these inputs with low error. However, when presented with anomalous or unseen abnormal data, the reconstruction error becomes significantly higher. This property makes autoencoders highly effective for anomaly detection tasks.

3. Materials/Tools:

- Python 3.x, TensorFlow/Keras, NumPy, Matplotlib
- Dataset: Normal vs. anomalous samples (e.g., MNIST digits with anomalies, network traffic data, or custom dataset)

4. Procedure (Pseudocode/Algorithmic Format)

Step 1: Import required libraries (TensorFlow/Keras, NumPy, Matplotlib).

Step 2: Load and preprocess the dataset, separating **normal data** (for training) and **test data** containing both normal and anomalous samples.

Step 3: Define the autoencoder architecture:

- Encoder: Dense/Conv layers to reduce dimensionality
- Decoder: Dense/Conv layers to reconstruct input

Step 4: Compile the autoencoder with optimizer (e.g., Adam) and loss function (e.g., Mean Squared Error).

Step 5: Train the autoencoder only on **normal data** for several epochs.

Step 6: Reconstruct test data and calculate reconstruction error for each sample.

Step 7: Define a **threshold value** for reconstruction error based on training/validation distribution.

Step 8: Classify samples as:

- **Normal:** error $\leq$ threshold
- **Anomaly:** error $>$ threshold

Step 9: Evaluate model performance by comparing predicted anomalies with ground truth.

Step 10: Visualize results (error distribution, reconstruction of normal vs. anomalous samples).

5. Results

- An autoencoder model was successfully implemented for anomaly detection.
- The model accurately reconstructed normal samples with low error, while anomalous samples showed significantly higher reconstruction error.
- By setting a threshold, the autoencoder was able to classify inputs as **normal** or **anomalous**.
- This experiment demonstrated how autoencoders can be applied effectively for detecting anomalies in datasets across domains like finance, cybersecurity, and healthcare.